IN512 – Exercice de mutation testing

Programme étudié : `Boutique.java` — calculateur de prix pour une boutique en ligne

Outil : Pitest (`pitest-maven` 1.15.8) via Maven, sous Eclipse

Équipe : *[Prénom NOM, Prénom NOM, ...] — (à compléter)*

Date : 14 octobre 2026

---

1. Présentation du programme

`Boutique.java` est une classe unique de 4 méthodes publiques (77 lignes avec commentaires Javadoc, 47 lignes de code effectif, aucune dépendance externe) :

- `calculerPrix(prixUnitaire, quantite, moisFidelite)` : applique une remise quantité (-5 % à partir de 3 articles, -10 % à partir de 10) puis une remise fidélité (-3 % à partir de 12 mois d'ancienneté, -7 % à partir de 24 mois).
- `fraisDePort(totalCommande, poidsGrammes)` : livraison gratuite à partir de 50 €, sinon 2,99 € si le colis pèse moins de 500 g, 4,99 € sinon.
- `niveauFidelite(moisAnciennete)` : renvoie `"bronze"`, `"argent"` ou `"or"` selon l'ancienneté (frontières à 12 et 24 mois).
- `pointsFidelite(montant, niveau)` : points de fidélité avec un bonus x1,5 (argent) ou x2 (or).

Le programme est volontairement petit mais dense en *frontières* numériques (`>= 3` , `>= 10` , `>= 12` , `>= 24` , `>= 50.0` , `< 500`) et en opérations arithmétiques (multiplications, soustractions, divisions implicites via `Math.round` / `Math.floor`). Complexité cyclomatique : `calculerPrix` = 5, `fraisDePort` = 3, `niveauFidelite` = 3, `pointsFidelite` = 3 (total ≈ 14 sur 4 méthodes). C'est un bon candidat pour la mutation : chaque frontière et chaque opérateur arithmétique est une source naturelle de mutants non équivalents, sans la complexité (boucles, structures de données) qui rend l'analyse d'un gros programme difficile à présenter en 4 pages.

---

2. Suite de tests préexistante

Nous avons écrit nous-mêmes une première suite, `BoutiqueTest.java` , dans l'esprit d'une suite « naïve » rédigée avant de connaître les résultats de mutation : **8 tests**, un par cas « évident » de chaque méthode (un cas de `calculerPrix` sans remise, un avec remise quantité, un avec remise fidélité ; les trois cas de `fraisDePort` ; le cas bronze de `niveauFidelite` et de `pointsFidelite`). Aucun de ces tests ne vise une valeur limite précise (ex. `quantite = 10` exactement), et les branches « argent »/« or » de `niveauFidelite` et `pointsFidelite` ne sont pas du tout exercées.

Cette suite est fonctionnellement correcte (les 8 tests passent) et donne une **couverture de lignes de 80 % (24/30 lignes mutées)**, ce qui semble correct à première vue — c'est précisément l'écart entre couverture de lignes et qualité réelle des tests que l'exercice de mutation doit révéler.

---

3. Méthodologie

- **Outil :** `org.pitest:pitest-maven` 1.15.8, plugin JUnit 5 (`pitest-junit5-plugin` 1.2.1), lancé via `mvn org.pitest:pitest-maven:mutationCoverage` .
- **Cibles :** `targetClasses = boutique.*` , `targetTests = boutique.*` .

- **5 mutateurs configurés explicitement** (choisis pour coller aux frontières et aux calculs du programme, plutôt que le groupe `DEFAULTS` complet) :
  - `CONDITIONALS_BOUNDARY` — décale les opérateurs relationnels d'une unité (`>=` → `>`, `<` → `<=`…), cible directe des 6 frontières du programme.
  - `NEGATE_CONDITIONALS` — inverse une condition (`>=` → `<`…).
  - `MATH` — remplace un opérateur arithmétique par un autre (`*` ↔ `/`, `-` ↔ `+`).
  - `INCREMENTS` — mute les opérateurs `++` / `--`.
  - `PRIMITIVE_RETURNS` — remplace une valeur de retour primitive par une valeur « neutre » (0, 0.0, false). *(Remplace `RETURN_VALS`, mutateur historique retiré des versions récentes de Pitest.)*
- Rapport généré au format HTML dans `target/pit-reports/`.

---

4. Résultats de l'analyse de mutation

Run initial (`BoutiqueTest.java` seul, 8 tests) :

Mutateur	Générés	Tués	Survivants
CONDITIONALS_BOUNDARY	8	0	8
NEGATE_CONDITIONALS	10	10	0
MATH	8	7	1
PRIMITIVE_RETURNS	4	4	0
INCREMENTS	0	—	—
Total	**30**	**21**	**9**

Score de mutation initial : 21/30 = 70 % (couverture de lignes 80 %).

`INCREMENTS` ne génère aucun mutant : le code ne contient aucun opérateur `++` / `--` (aucune boucle). C'est un résultat en soi : un mutateur bien choisi en théorie peut être totalement inopérant selon le style d'écriture du code cible — à vérifier avant de le configurer sur un vrai projet.

Les **9 mutants survivants** sont exactement les frontières et le calcul non exercés par la suite naïve :

1. `calculerPrix` : `quantite >= 10` (frontière non testée pile à 10)
2. `calculerPrix` : `quantite >= 3` (frontière non testée pile à 3)
3. `calculerPrix` : `moisFidelite >= 24`
4. `calculerPrix` : `moisFidelite >= 12`
5. `fraisDePort` : `totalCommande >= 50.0`
6. `fraisDePort` : `poidsGrammes < 500`
7. `niveauFidelite` : `moisAnciennete >= 24`
8. `niveauFidelite` : `moisAnciennete >= 12`
9. `pointsFidelite` : `montant * bonus` (mutant `MATH`, multiplication ↔ division) — invisible avec le seul test bronze, où `bonus = 1.0` rend les deux opérations indiscernables (`x * 1.0 == x / 1.0`).

Tests ajoutés (`BoutiqueTestAjoutes.java`, 16 tests), construits par analyse de frontière (*boundary value analysis*) : pour chaque mutant survivant, un test à la valeur exacte de la frontière, et pour la plupart un second test juste en dehors (ex. `quantite = 10` et `quantite = 9` ; `moisFidelite = 12` et

`11`…). Pour le mutant `MATH` de `pointsFidelite`, deux tests avec `bonus ≠ 1.0` (niveaux « argent » et « or ») suffisent à rendre la mutation observable.

Run final (24 tests, 8 + 16) :

Mutateur	Générés	Tués	Survivants
CONDITIONALS_BOUNDARY	8	8	0
NEGATE_CONDITIONALS	10	10	0
MATH	8	8	0
PRIMITIVE_RETURNS	4	4	0
Total	**30**	**30**	**0**

Score de mutation final : 30/30 = 100 %, couverture de lignes 100 % (30/30), 65 exécutions de tests pour 30 mutants (~2,2 tests pertinents par mutant, grâce à l'analyse de couverture de Pitest qui n'exécute que les tests touchant réellement le code muté).

---

5. Avantages et inconvénients constatés

Avantages. La mutation a mis en évidence un défaut invisible à la couverture de lignes seule : les 8 tests initiaux couvraient déjà 80 % des lignes, un chiffre qu'une revue de couverture classique aurait jugé correct, alors que *aucune* frontière n'était testée précisément — le genre d'erreur « off-by-one » le plus fréquent en pratique. Le rapport HTML de Pitest localise chaque mutant survivant ligne par ligne, ce qui a rendu la correction mécanique une fois le principe compris (une frontière = un test à la valeur exacte). Le mutant `MATH` sur `pointsFidelite` illustre bien l'hypothèse de « l'effet de couplage » du cours : un test insuffisamment varié (toujours `bonus = 1.0`) masque une mutation sur une opération qui serait pourtant visible avec d'autres entrées.

Inconvénients. Le coût de calcul reste faible ici (4 secondes, 30 mutants, 65 exécutions de tests) précisément parce que le programme est petit ; sur un projet réel de plusieurs milliers de lignes, le temps d'exécution croît vite (un run complet de la suite de tests par mutant). Nous n'avons rencontré aucun mutant équivalent dans ce cas précis — attendu vu la taille du programme et l'absence de code mort ou redondant — mais le risque reste réel dès que le code grossit, et rien ne garantit a priori qu'un mutant survivant soit « tuable » : il faut l'inspecter manuellement pour le savoir, ce qui ne s'automatise pas. Enfin, le choix des mutateurs n'est pas neutre : `INCREMENTS` ici n'a rien détecté, uniquement parce que le code ne contenait pas l'opérateur ciblé — un mauvais point de départ pour juger un score de mutation sans regarder le détail par mutateur.

---

6. Ce que cet exercice nous a appris sur la mutation

L'enseignement principal est que **la couverture de lignes et la couverture de mutation mesurent deux choses différentes**, et que la première peut masquer complètement l'absence de la seconde. Nos 8 tests initiaux atteignaient 80 % de couverture de lignes — un score qu'on présenterait sans hésiter comme « suffisant » dans un contexte classique — alors que le score de mutation n'était que de 70 %, et surtout, les 9 mutants survivants n'étaient pas répartis au hasard : ils correspondaient *tous* aux frontières numériques du programme, exactement le type de faute (`>=` écrit `>`, ou l'inverse) qu'un programmeur compétent mais pressé commet le plus souvent. C'est une illustration concrète de l'hypothèse du « programmeur compétent » du cours : les mutants ne sont pas des fautes artificielles

exotiques, ce sont des fautes réalistes, et un test qui ne les détecte pas ne détecterait probablement pas la vraie faute équivalente non plus.

Le deuxième enseignement concerne la méthode pour *réagir* à un score de mutation insuffisant : il ne s'agit pas d'ajouter des tests au hasard, mais de lire le rapport mutant par mutant et de construire, pour chacun, le test qui sépare le comportement original du comportement muté — typiquement une valeur exactement sur la frontière concernée. Cette démarche rejoint directement l'analyse aux limites (*boundary value analysis*) vue en cours avec l'exemple du triangle de Myers : la mutation ne fait que formaliser et automatiser la détection des frontières non testées, là où l'exercice du triangle demandait de les anticiper « à la main ». Avoir un outil qui les révèle *après coup*, par l'exécution, est beaucoup plus fiable qu'une relecture de code pour s'assurer qu'aucune frontière n'a été oubliée.

Nous avons aussi mesuré une limite pratique de l'approche : passer de 70 % à 100 % nous a coûté 16 tests supplémentaires pour un programme de seulement 47 lignes de code utile. Le ratio tests/code nécessaire pour un score élevé monte vite, et sur un programme plus gros, viser 100 % de score de mutation deviendrait rapidement un objectif coûteux à atteindre pour un gain marginal décroissant — surtout face à la possibilité de mutants équivalents indécidables, que nous n'avons pas rencontrés ici mais qui existent dès que le code comporte de la redondance ou des conditions logiquement équivalentes. Le score de mutation nous semble donc un excellent outil de *diagnostic* pour savoir où ajouter des tests, mais un mauvais objectif chiffré à atteindre à 100 % de façon systématique sur un vrai projet : il vaut mieux l'utiliser pour prioriser l'effort de test sur le code critique que comme une métrique de qualité globale à maximiser aveuglément.

Code source, tests et rapports Pitest (`target/pit-reports/`) disponibles dans l'archive jointe. Projet Maven, compilable avec `mvn test` et analysable avec `mvn org.pitest:pitest-maven:mutationCoverage`.